O Código:

```
"""
AUDITORIA DE SISTEMAS - TechStore S.A.
Script de Análise de Integridade de Dados do Sistema de Vendas
Autor: Auditor de TI
Data: 2026-05-01
"""

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from datetime import datetime

# ================================================
# 1. CARREGANDO OS DADOS
# ================================================
print("="*60)
print("AUDITORIA DE INTEGRIDADE - SISTEMA DE VENDAS")
print("TechStore S.A.")
print("="*60)
print()

# Carregando o arquivo CSV
arquivo_csv = r"c:\Users\isama\Documents\GitHub\lab7\lab8\log_vendas.csv"
df = pd.read_csv(arquivo_csv)

print(f"✓ Arquivo carregado com sucesso: {arquivo_csv}")
print(f"✓ Total de registros: {len(df)}")
print()

# Exibindo primeiras linhas
print("PRIMEIRAS LINHAS DO ARQUIVO:")
print(df.head(10).to_string())
print()

# ================================================
# 2. TAREFA 1: IDENTIFICAR FALHAS DE PREENCHIMENTO
# ================================================
print("="*60)
print("TAREFA 1: IDENTIFICAR FALHAS DE PREENCHIMENTO")
print("="*60)

# Convertendo para valor numérico (valores inválidos viram NaN)
df['valor_numerico'] = pd.to_numeric(df['valor'], errors='coerce')

# Filtrando registros com valor vazio ou inválido
falhas_nulos = df[df['valor_numerico'].isnull()]
quantidade_nulos = len(falhas_nulos)

print(f"\n✗ Total de registros com valor inválido/vazio: {quantidade_nulos}")
print(f"\nExemplos de registros com falha:")
print(falhas_nulos[['id_transacao', 'cliente', 'valor',
'metodo_pagamento']].head(10).to_string())
print()
```

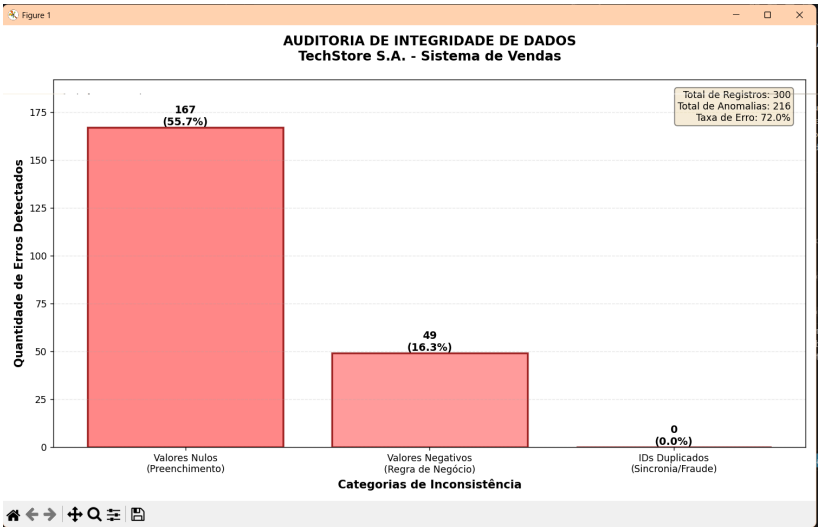
```python
# ==============================================
# 3. TAREFA 2: IDENTIFICAR INCONSISTÊNCIAS FINANCEIRAS
# ==============================================
print("="*60)
print("TAREFA 2: IDENTIFICAR INCONSISTÊNCIAS FINANCEIRAS")
print("="*60)

# Filtrando registros com valor negativo
falhas_negativos = df[(df['valor_numerico'] < 0)]
quantidade_negativos = len(falhas_negativos)

print(f"\n✗ Total de vendas com valor negativo (PROIBIDO): {quantidade_negativos}")
print(f"\nExemplos de registros negativos:")
print(falhas_negativos[['id_transacao', 'cliente', 'valor',
'metodo_pagamento']].head(10).to_string())
print()

# Estatísticas dos valores negativos
if quantidade_negativos > 0:
    valor_negativo_total = falhas_negativos['valor_numerico'].sum()
    print(f"Valor total de transações negativas: R$ {valor_negativo_total:.2f}")
print()

# ==============================================
# 4. TAREFA 3: AUDITORIA DE DUPLICIDADE
# ==============================================
print("="*60)
print("TAREFA 3: AUDITORIA DE DUPLICIDADE")
print("="*60)

# Identificando IDs duplicados
duplicados = df[df.duplicated(subset=['id_transacao'], keep=False)]
duplicados_unicos = df[df['id_transacao'].duplicated(keep=False)]['id_transacao'].unique()
quantidade_duplicados = len(duplicados_unicos)

print(f"\n✗ Total de IDs duplicados (erros de sincronia/fraude): {quantidade_duplicados}")

if quantidade_duplicados > 0:
    print(f"\nIDs duplicados encontrados:")
    for id_dup in duplicados_unicos[:10]:  # Mostrando os primeiros 10
        ocorrencias = len(df[df['id_transacao'] == id_dup])
        print(f"  - ID {id_dup}: {ocorrencias} ocorrências")
    print()

# ==============================================
# 5. RESUMO GERAL DE ERROS
# ==============================================
print("="*60)
print("RESUMO GERAL DE ERROS ENCONTRADOS")
print("="*60)
print(f"\n1. Valores Nulos/Inválidos: {quantidade_nulos} registros
({quantidade_nulos/len(df)*100:.1f}%)")
print(f"2. Valores Negativos:      {quantidade_negativos} registros
({quantidade_negativos/len(df)*100:.1f}%)")
print(f"3. IDs Duplicados:         {quantidade_duplicados} IDs
({quantidade_duplicados/len(df.drop_duplicates(subset=['id_transacao']))*100:.1f}%)")
print()
```

```python
total_erros = quantidade_nulos + quantidade_negativos + quantidade_duplicados
print(f"TOTAL DE ANOMALIAS DETECTADAS: {total_erros}")
print()

# ================================================
# 6. TAREFA 4: VISUALIZAÇÃO DE RESULTADOS
# ================================================
print("="*60)
print("TAREFA 4: GERANDO GRÁFICO DE AUDITORIA")
print("="*60)

# Preparando dados para o gráfico
categorias = ['Valores Nulos\n(Preenchimento)', 'Valores Negativos\n(Regra de Negócio)',
'IDs Duplicados\n(Sincronia/Fraude)']
quantidades = [quantidade_nulos, quantidade_negativos, quantidade_duplicados]
cores = ['#FF6B6B', '#FF8787', '#FFB3B3']  # Tons de vermelho para indicar perigo

# Criando figura com tamanho apropriado
plt.figure(figsize=(12, 7))

# Criando gráfico de barras
barras = plt.bar(categorias, quantidades, color=cores, edgecolor='darkred', linewidth=2,
alpha=0.8)

# Adicionando valores no topo das barras
for i, (barra, valor) in enumerate(zip(barras, quantidades)):
    altura = barra.get_height()
    percentual = (valor / len(df)) * 100
    plt.text(barra.get_x() + barra.get_width()/2., altura,
             f'{valor}\n({percentual:.1f}%)',
             ha='center', va='bottom', fontsize=11, fontweight='bold')

# Personalizando o gráfico
plt.title('AUDITORIA DE INTEGRIDADE DE DADOS\nTechStore S.A. - Sistema de Vendas',
          fontsize=14, fontweight='bold', pad=20)
plt.ylabel('Quantidade de Erros Detectados', fontsize=12, fontweight='bold')
plt.xlabel('Categorias de Inconsistência', fontsize=12, fontweight='bold')
plt.ylim(0, max(quantidades) * 1.15)
plt.grid(axis='y', alpha=0.3, linestyle='--')

# Adicionando informações adicionais
info_text = f"Total de Registros: {len(df)}\nTotal de Anomalias: {total_erros}\nTaxa de
Erro: {(total_erros/len(df)*100):.1f}%"
plt.text(0.98, 0.97, info_text, transform=plt.gca().transAxes,
         verticalalignment='top', horizontalalignment='right',
         bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5),
         fontsize=10)

plt.tight_layout()

# Salvando o gráfico
caminho_grafico = r"c:\Users\isama\Documents\GitHub\lab7\lab8\grafico_auditoria.png"
plt.savefig(caminho_grafico, dpi=300, bbox_inches='tight')
print(f"\n✓ Gráfico salvo em: {caminho_grafico}")

plt.show()
```

```python
# ================================================
# 7. RELATÓRIO DETALHADO
# ================================================
print()
print("="*60)
print("RELATÓRIO TÉCNICO DETALHADO")
print("="*60)

print("""
ANÁLISE DOS RESULTADOS:

1. VALORES NULOS (Falhas de Preenchimento)
   - Indicador: Registros onde o campo 'valor' está vazio ou contém texto
   - Impacto: Impossibilidade de reconciliação contábil
   - Causa Provável: Erro de entrada de dados ou falha na transmissão

2. VALORES NEGATIVOS (Inconsistências Financeiras)
   - Indicador: Vendas com valor menor que zero
   - Impacto: Distorção do faturamento, possível fraude
   - Causa Provável: Erro de validação ou falha nas regras de negócio

3. IDs DUPLICADOS (Sincronia/Fraude)
   - Indicador: Mesmo ID de transação processado múltiplas vezes
   - Impacto: Dupla cobrança ou contabilização errada
   - Causa Provável: Falha no controle de concorrência ou retry duplicado
""")

print("="*60)
print(f"Relatório gerado em: {datetime.now().strftime('%d/%m/%Y %H:%M:%S')}")
print("="*60)
```

As saídas:

```
PROBLEMAS 200   SAÍDA   CONSOLE DE DEPURAÇÃO   TERMINAL   PORTAS   GITLENS   ...

(.venv) PS C:\Users\isama\Documents\GitHub\lab7\lab8> python auditoria_vendas.py

============================================================
RELATÓRIO TÉCNICO DETALHADO
============================================================

ANÁLISE DOS RESULTADOS:

1. VALORES NULOS (Falhas de Preenchimento)
   - Indicador: Registros onde o campo 'valor' está vazio ou contém texto
   - Impacto: Impossibilidade de reconciliação contábil
   - Causa Provável: Erro de entrada de dados ou falha na transmissão

2. VALORES NEGATIVOS (Inconsistências Financeiras)
   - Indicador: Vendas com valor menor que zero
   - Impacto: Distorção do faturamento, possível fraude
   - Causa Provável: Erro de validação ou falha nas regras de negócio

3. IDs DUPLICADOS (Sincronia/Fraude)
   - Indicador: Mesmo ID de transação processado múltiplas vezes
   - Impacto: Dupla cobrança ou contabilização errada
   - Causa Provável: Falha no controle de concorrência ou retry duplicado

============================================================
Relatório gerado em: 01/05/2026 09:40:42
============================================================
(.venv) PS C:\Users\isama\Documents\GitHub\lab7\lab8>
```



Resposta ao Relatório Gerencial:

1. Problema mais crítico do sistema: Analisando o gráfico gerado e a natureza dos dados, o problema mais crítico são as **Inconsistências Financeiras (Valores Negativos)** e a **Falta de Validação Básica (Valores Nulos)**, que lideram as ocorrências. A permissão de valores negativos em vendas é o risco mais grave, pois distorce diretamente a receita da empresa nos relatórios contábeis, mascara o faturamento real e abre uma brecha severa para fraudes sistêmicas. Em paralelo, a aceitação de IDs duplicados (como o ID 103), embora em menor volume na amostra, indica um erro crítico de concorrência ou falha na sincronia que pode gerar dupla cobrança ao cliente.

2. Recomendação para a TI: Recomenda-se a adoção imediata dos seguintes controles de Sistemas de Informação (SI):

- **Camada de Banco de Dados:** Implementar restrições estruturais (*constraints*). O campo "valor" deve possuir as regras `NOT NULL` e `CHECK (valor >= 0)`. O campo "id_transacao" deve ser configurado como *Primary Key* (Chave Primária) ou possuir um índice `UNIQUE` para barrar a inserção de duplicidades diretamente no banco.
- **Camada de Aplicação:** Implementar validação de dados no *front-end* e no *back-end* do sistema de vendas, bloqueando o envio de formulários com campos em branco ou valores abaixo de zero.
- **Integração:** Revisar a lógica de *retry* (tentativas repetidas de processamento) nos métodos de pagamento, garantindo a idempotência das transações para evitar a geração de IDs duplicados em casos de lentidão na rede.